

Autonomous Software Agents: Deliveroo.js

Davide Donà

`davide.dona-1@studenti.unitn.it` Andrea Blushi

`andrea.blushi@studenti.unitn.it`

University of Trento

June 2026



UNIVERSITÀ
DI TRENTO

Contents

- 1 Introduction
- 2 BDI Agent
- 3 PDDL Crate Navigation
- 4 LLM Agent Layer

Our Solution

- **BDI core** - reactive collection, exploration
- **PDDL planner** - crate-clearing
- **LLM layer** - high-level control via structured tools
- **Cooperation** — coordination via peer communication

Execution Pipeline

- One cycle per sensing socket event
- **Perceive** → **Deliberate** → **Plan** → **Execute**
- One action emitted per server clock tick
- Re-deliberates after each successful move

[agent loop]

Beliefs: Tracker & Memory

- **Tracker** — latest observation $\langle v, t_{seen} \rangle$ per entity
- Confidence decays: $c(t) = e^{-(t-t_{seen})/\tau}$
- **Memory** — bounded temporal history per entity
- Observations invalidated when visible-but-absent

Beliefs: Map

- Static tile layout, transient blocked tiles, traversal penalties
- *Sink tiles*: agent can enter but never complete a loop
- Modelled as directed walkability graph (conveyors = one-way edges)
- Tarjan SCC — only tiles in safe SCCs remain walkable

Beliefs: Parcels & Crates

- Parcel reward decays linearly (-1 per server tick) when out of sight
- $r(t) = r_0 - \lfloor (t - t_0) / \Delta_t \rfloor$
- Parcel removed when $r(t) \leq 0$
- Crate positions tracked; invalidated once pushed out of view

Desires

- **ReachParcel** — navigate to and collect a visible parcel
- **DeliverParcel** — bring carried parcels to delivery tile
- **Explore** — seek unvisited spawn tile when idle
- Parcel goals (priority 1) always beat Explore (priority 0)
- Intention queue fully rebuilt each deliberation cycle

Scoring: Parcels

ReachParcel

$$S = \frac{r_p + r_{carried}}{d} \cdot \gamma, \quad \gamma = \min\left(1, \frac{d_{enemy}}{d}\right)$$

Race factor γ de-prioritises parcels contested by enemies.

DeliverParcel

$$S = \frac{r_{carried}}{d}$$

Scoring: Explore

- w — spawn-cluster density (nearby spawn tiles)
- τ — freshness (time since tile was last observed)
- h — enemy heat (spatial + temporal Gaussian decay)
- $S = \frac{w \cdot \tau}{d \cdot (1 + h)}$

Planning: A*

- Manhattan distance heuristic (admissible on grid)
- Walkability checked dynamically from beliefs
- Conveyors: directional movement only
- Plan reused while desire type and target unchanged

[pathfinding]

Collision Handling

Next tile occupied? Three-stage escalation:

- ➊ **Detour** — replan with tile as wall (within cost threshold)
- ➋ **Wait** — random pause duration
- ➌ **Block & replan** — temporary TTL block, full replan

[collision resolution]

When A* Fails

- A* fails: no path that avoids crates
- Second A* ignoring crates finds a “ghost” path
- Crates intersecting that path confirm blockage
- PDDL planner invoked as fallback

[crate push]

The PDDL Domain

- STRIPS: two object types — tile and crate
- 4 directional **move** actions (step into crate-free tile)
- 4 directional **push** actions:
 - Agent adjacent on opposite side of push direction
 - Destination must be a valid, crate-free tile
- Push relocates crate; agent steps onto its former tile

Performance Bottleneck

- PDDL solver too slow for real-time play
- Moved to local solver: eliminated remote API overhead
- Caching fails — agent position changes the cache key
- Trivial position shift $\Rightarrow$ full re-solve of same maneuver

Smart Fix: Problem Restatement

Split the journey into three segments

- ➊ A* from current position to cluster *entry*
 - ➋ PDDL through the crate cluster (*entry* → *exit*)
 - ➌ A* from cluster *exit* to final goal
- PDDL problem anchored to cluster entry tile
 - Cache key: (*entry*, *exit*, {crate positions})

LLM Layer Overview

- Thin wrapper around the BDI agent
- Listens for mission-controller chat messages
- Up to 4 tool-call rounds (hops) per message
- Three command categories: **goals** · **rules** · **coordination**

Goal Injection

- **go-to** → REACH_TILE desire (configurable reward + TTL)
- **deliver-at** → DELIVER_PARCEL at specific tile
- Both compete with parcel goals via standard scoring
- Persist and re-evaluated until executed or TTL expires

Rule Injection

- RuleStore: condition + multiplier m + offset a
- Applied each cycle: scales/shifts baseline desire scores
- Example: incentivise collecting n parcels before delivery
- Stack rule automatically elevates Explore to priority 1

Traversal Penalties

- Adds additive cost to A^* edge leading onto a tile
- Agent prefers detour if $\text{detour} < \text{penalty length}$
- Tile not forbidden — used as last resort
- Operates at the planning layer, not the deliberation layer

Team Communication

- **Beacons** — each agent broadcasts position at fixed interval
- Ensures up-to-date peer positions beyond sensing radius
- **Peer-injection** — command applied locally, then broadcast
- Simple commands (goals, rules, penalties) are fire-and-forget

Holds: 2-Phase Commit

`request_rendezvous` &
`request_red_light` use 2PC:

- **Phase 1** — propose broadcast; each teammate votes
- Vote: accept if hold-tile score beats current best goal
- **Phase 2** — unanimous accept → commit; else abort
- Red light: hold on nearest odd-row tile; green light clears all holds

[traffic control]

Coordinator

- Periodic rounds: broadcast `request_beliefs` to team
- Collects positions, payloads, high-value zones
- LLM receives geometric summary → issues `assign_goto` commands
- Distributes agents across the map optimally
- Also triggered on-demand via `request_team_status`

